



SYSTEM ANALYSIS DOCUMENTATION(SAD)

SmartRent AI (Decision Intelligence Engine)



APRIL 25, 2026

Muhammad Aidil Bin Mohd Alias
Tania Danisha Puteri Binti Mohd Shafie

Table Of Content

1. Introduction	2
1.1. Purpose	2
1.2. Background	2
1.3. Target Stakeholders	3
2. System Architecture & Design	3
2.1. High Level Architecture	3
2.1.1. Overview	3
2.1.2. LLM as Service Layer	3
2.1.3 Dependency Diagram	5
2.1.4 Sequence Diagram: Rental Analysis & Ranking Flow	6
2.2. Technological Stack	8
2.3 Key Data Flaws	9
2.3.1. Data Flow Diagram (DFD)	9
2.3.2 Normalized Database Schema (Logical ERD)	10
3. Functional Requirements & Scope	13
3.1. Minimum Viable Product (MVP)	13
3.2. Non-Functional Requirements (NFRs)	13
3.3. Out of Scope / Future Enhancements	14
4.1. Technical Evaluation	14
4.1.1. Grayscale Rollout & A/B Testing	14
4.1.2. Emergency Rollback (ER) & Golden Release	14
4.1.3. Priority Matrix	15
4.2. Monitoring	15
4.3. Assumptions	15
4.4. External Dependencies	15
5. Project Management & Team Contributions	16
5.1. Project Timeline:	16
5.2 Team Members Role:	16
5.3. Recommendations	17

SYSTEM ANALYSIS DOCUMENTATION

Project Name: SmartRent AI (Decision Intelligence Engine)

Version: 13.0

1. Introduction

This section introduces SmartRent AI as the strategic solution to the problem of "economic blindness" identified in the Product Requirement Document. The system focuses on empowering users to make data-driven housing decisions through high-fidelity visualization and AI reasoning.

1.1. Purpose

The system analysis document highlights the technical scope and design-related decisions behind the development of SmartRent AI. Key elements of the system covered in this document are as follows:

- **Architecture:**

The platform is built as an AI-native frontend system that integrates Vercel Serverless Functions as the core relay for the Z.AI GLM service layer.

- **Data Flows:**

Information such as salary, commute logs, and rental ads flow from the user's browser to the Z.AI GLM API and return as a ranked comparison on the visual canvas.

- **Model Process:**

The system demonstrates an end-to-end workflow for parsing unstructured rental data, calculating hidden costs, and recommending the best housing options.

- **Role of Reference:**

This documentation serves as a technical guide for developers and judges to understand the high-level architecture and logic of the project.

1.2. Background

Market research shows that the current process of searching for rental rooms and evaluating financial health is highly manual and fragmented. Many fresh graduates and urban workers struggle to see their true disposable income because they lack the tools to visualize hidden costs like transportation and deposits. SmartRent AI replaces this manual guesswork with an automated, AI-driven platform that optimizes the housing selection process.

1.3. Target Stakeholders

The following table identifies the key individuals involved and their specific expectations for the system:

Stakeholders	Roles	Expectations
End User	Inputs personal financial data and rental advertisements to receive a ranked list.	A simple, fast experience that helps identify the most cost-effective housing.
Developer Team	Builds the frontend interface and manages the integration of the Z.AI GLM API.	Modular and well-documented codebase for easy debugging and future updates.
QA Team / Judges	Validates system behaviour, logic accuracy, and handles edge cases for messy ads.	Clearly defined logic for financial reasoning and stable performance under different scenarios.

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview

Type	Details
System	Serverless-Relay Architecture
Architecture	Web-based AI Prototype (Vanilla JS & Node.js Functions)

SmartRent AI is structured as a frontend-driven system that utilizes Vercel Functions as a serverless relay to connect to the Z.AI GLM API. This architecture is intentionally stateless to ensure the application remains lightweight and fast for the hackathon demonstration. The system captures user financial data and rental advertisements within the browser and channels them through the serverless relay as prompts to the AI engine for real-time processing.

2.1.2. LLM as Service Layer

Z.AI GLM is integrated as the primary service layer of the architecture rather than a simple chatbot module. This layer functions as the central "reasoning engine" that replaces traditional backend processing for decision intelligence. Communication with this layer is managed via a Node.js-based Serverless Relay

to ensure secure API authentication and to bypass CORS limitations. The following responsibilities are handled by this layer:

- **Context Construction:** The system aggregates the user's salary, fixed financial commitments, travel logistics, and the latest chat history into a detailed prompt window.
- **Unstructured Data Parsing:** AI logic is applied to analyse messy text or links from rental advertisements to identify critical data such as monthly rent, deposits.
- **Token Management:** A dedicated chunking mechanism within the Serverless Relay ensures that all oversized advertisement inputs are trimmed before they reach the GLM to strictly stay within the 1500-token limit.
- **Reasoning & Ranking:** The model performs multi-step financial calculations to determine the "Real Cost of Living" and generates structured data for the visual ranking system.
- **Conversational Logic Editing:** This sub-module utilizes AI Intent Parsing via the Z.AI GLM API to process unstructured chat commands. It identifies the target (e.g., AD index or Profile) and the specific variable to be updated (e.g., salary, rent, or distance), enabling real-time logic adjustments.

2.1.3 Dependency Diagram

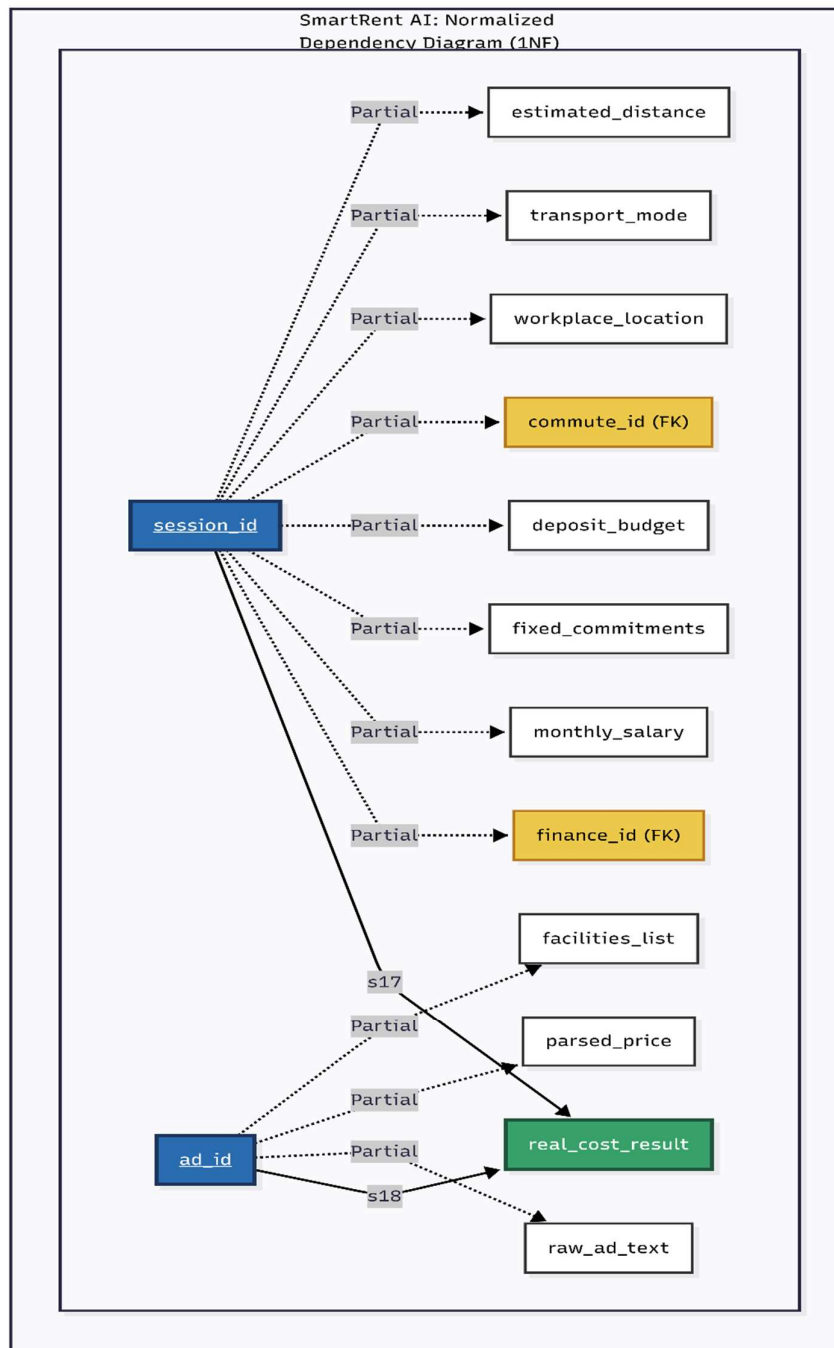


Figure 1: Dependency Diagram

Dependency Diagram illustrates the technical relationships and data requirements between the system components and the external AI service.

- **Prompt Construction:** SmartRent AI constructs reasoning prompts by aggregating the End User's Financial Profile, Commute Logistics, and raw Rental Ad Data into a single context-aware instruction.
- **Context Window Content:** The active context window includes the specific variables from the Financial Profile (salary and debts), transport preferences from Commute Logistics, and extracted features from the Rental Ad Data.
- **Response Parsing:** The system receives the raw completion from the Z.AI GLM API, parses the text into a structured JSON object, and passes the data to the Comparison Canvas for visual rendering.
- **Token Enforcement:** A dedicated Token Management Module chunks or truncates the Rental Ad Data before it reaches the Z.AI GLM API to ensure the total input remains within the 1500-token limit.
- **Major API Calls:** SmartRent AI makes a direct HTTPS POST request to the Z.AI GLM API to trigger the multi-step financial reasoning engine.
- **Component Interaction:** SmartRent AI manages the user interface and session state while interacting directly with the Z.AI GLM API, which removes the need for a traditional backend or RDS database in this prototype.
- **Deployment Strategy:** The application is deployed as a distributed frontend service on Vercel to provide the judges with high availability and a stable public link.

2.1.4 Sequence Diagram: Rental Analysis & Ranking Flow

This section illustrates the technical interactions for the two primary use cases of the system:

Flow A: Initial Prompt to Analysis

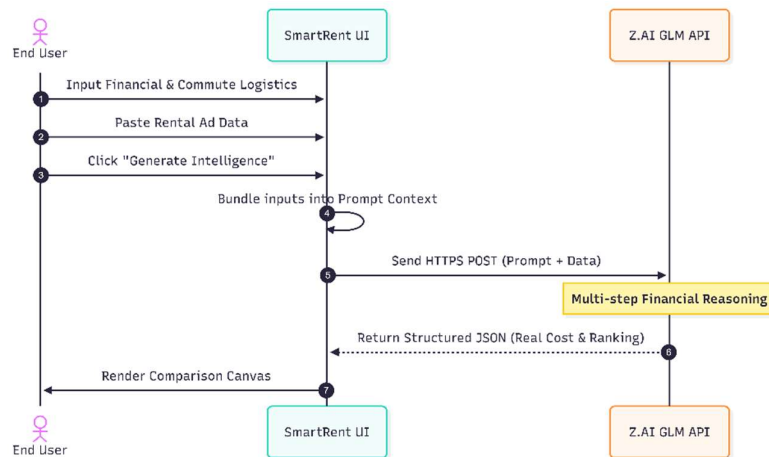


Figure 2: sequence diagram (Flow

This flow represents the first time an End User generates a housing analysis.

1. **Data Entry:** End User inputs the Financial Profile and Commute Logistics into the SmartRent AI.
2. **Ad Submission:** End User pastes the Rental Ad Data into the interface.
3. **Processing:** SmartRent AI bundles all inputs and sends a request to the Z.AI GLM API.
4. **AI Reasoning:** Z.AI GLM API performs multi-step calculations to determine the "Real Cost of Living".
5. **Data Return:** Z.AI GLM API sends the structured financial data back to the SmartRent AI.
6. **Rendering:** SmartRent AI displays the results on the Comparison Canvas.

Flow B: Multi-Option Ranking & Conversational Refinement

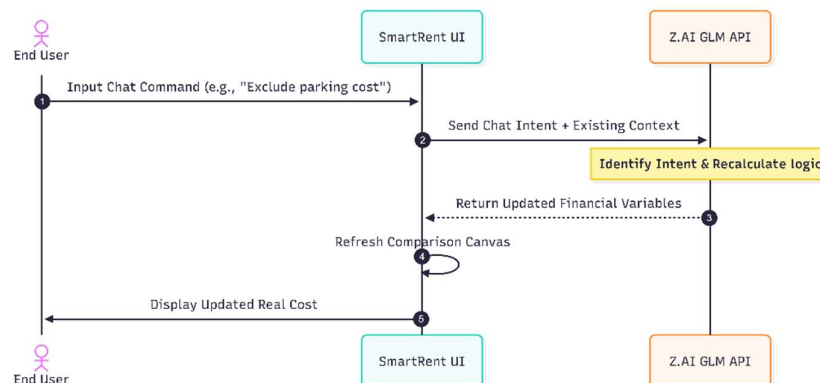


Figure 3: sequence diagram (Flow B)

1. **Trigger Comparison:** End User selects the "Add Another" option in the SmartRent AI.
2. **Multi-Ad Submission:** End User submits additional sets of Rental Ad Data (up to five total).
3. **Batch Processing:** SmartRent AI bundles the updated list of Rental Ad Data with the existing Financial Profile and Commute Logistics.
4. **Simultaneous Reasoning:** Z.AI GLM API processes all rental options simultaneously to ensure a consistent economic comparison.
5. **Ranked Output:** SmartRent AI renders the updated ranking list on the Comparison Canvas.
6. **Conversational Edit:** End User types a command (e.g., "Exclude car parking cost") into the chat interface.
7. **Logic Update:** Z.AI GLM API identifies the intent, recalculates the "Real Cost" for all active ads, and SmartRent AI refreshes the Comparison Canvas instantly.

2.2. Technological Stack

Stack Layer	Technology	Technical Justification
Frontend	Modern HTML5/JS.	Provides a responsive environment for the SmartRent AI. Tailwind CSS is utilized for the rapid styling of the 2D Comparison Canvas.
Backend	Stateless API Integration	The system utilizes a serverless approach, allowing the SmartRent AI to communicate directly with the Z.AI GLM API without a traditional server (PHP/Node.js).
Data Storage	React State (Hooks/Context)	Manages temporary session storage for the Financial Profile and Rental Ad Data within the browser's memory, ensuring zero-latency data access.
Cloud	Vercel	Serves as the primary deployment platform to provide a stable, high-performance public link for judges and stakeholders.

2.3 Key Data Flows

This section describes the movement and logical structure of data within SmartRent AI. Although the system is stateless, it follows rigorous data flow and normalization principles to ensure decision accuracy.

2.3.1. Data Flow Diagram (DFD)

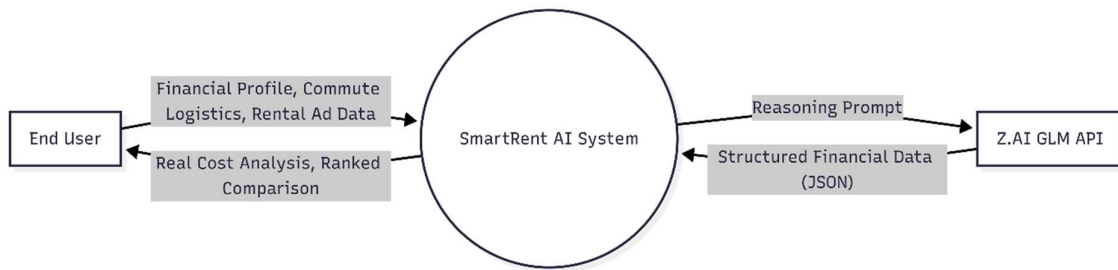


Figure 4: Context Diagram (DFD Level 0)

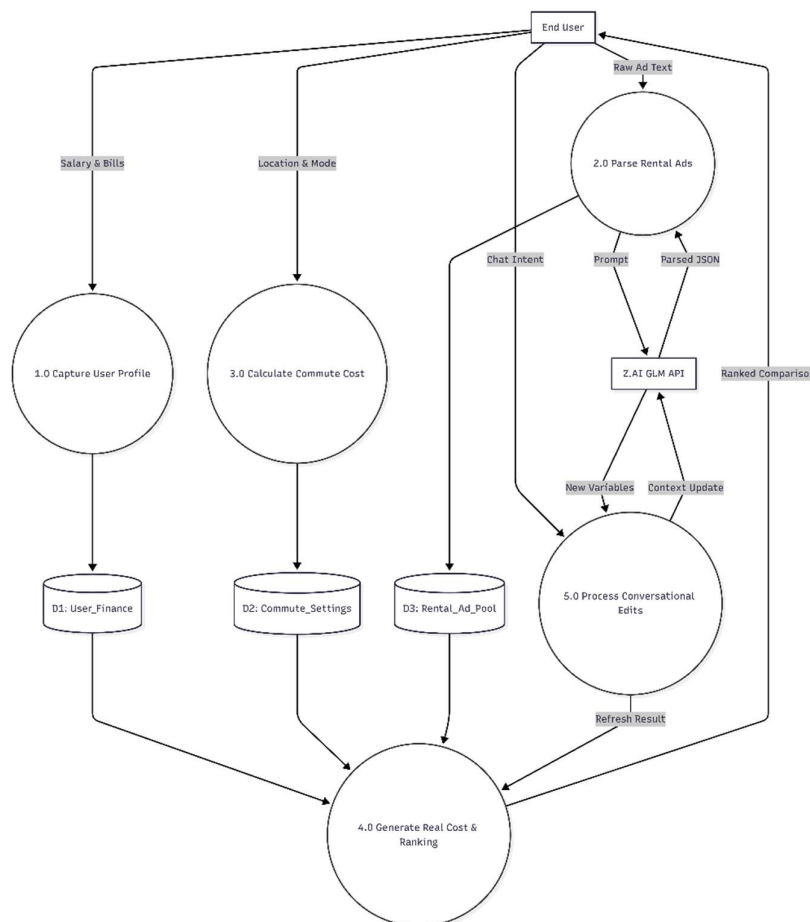


Figure 5: DFD Level 1

The Data Flow Diagram (DFD) illustrates how information moves between the End User, the SmartRent AI, and the Z.AI GLM API.

- **Entities & Processes:** Data originates from the End User and is processed by the SmartRent AI to be stored temporarily in the React Session State. This data is then channelled to the Z.AI GLM API for reasoning.
- **Data Integrity:** The flow ensures that Financial Profiles and Commute Logistics are paired with Rental Ad Data before being sent as a prompt.
- **Avoiding "Black Holes":** The system strictly adheres to data flow laws. Every input collected (e.g., salary or transport mode) is processed and transformed into a visible metric on the Comparison Canvas. There is no "Black Hole" in the system; all stored session data is utilized to generate the final economic ranking.

2.3.2 Normalized Database Schema (Logical ERD)

To maintain data integrity and prepare for future scalability, the system's logical data model is designed in Third Normal Form (3NF). This structure resolves many-to-many relationships and eliminates data redundancy.

Entity	Primary Key	Attributes	Description
User_Finance	finance_id	monthly_salary, fixed_commitments, deposit_budget	Stores the baseline financial constraints.
Commute_Settings	commute_id	workplace_location, transport_mode, estimated_distance	Stores transportation variables for logic updates.
Rental_Ad_Pool	ad_id	raw_ad_text, parsed_price, facilities_list	Stores the parsed details of various rental advertisements.
Comparison_Session	session_id	finance_id (FK), commute_id (FK), ad_id (FK), real_cost_result	The Junction Table that resolves the relationship between user constraints and multiple rental ads to produce the ranking.

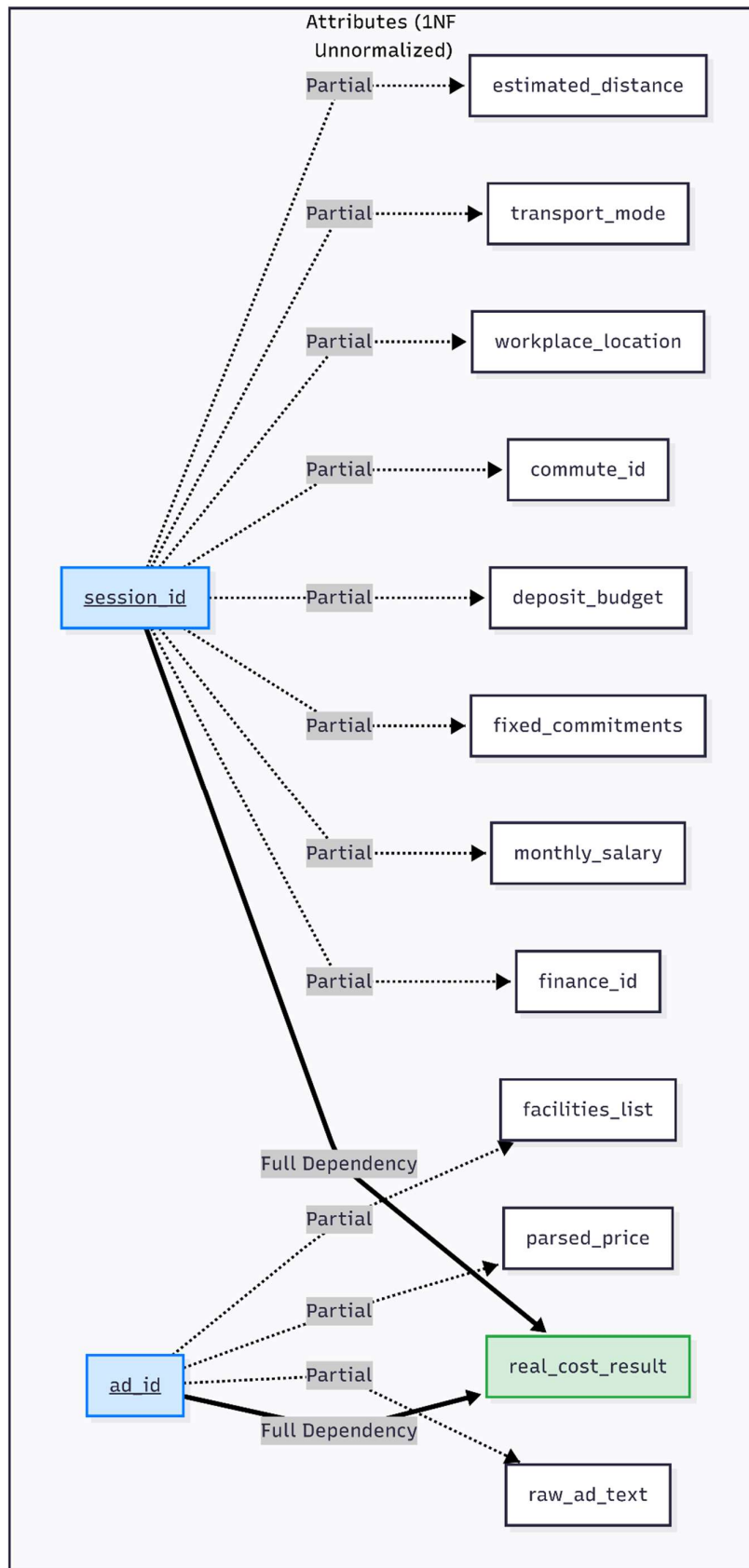


Figure 6: 1 Normal Form

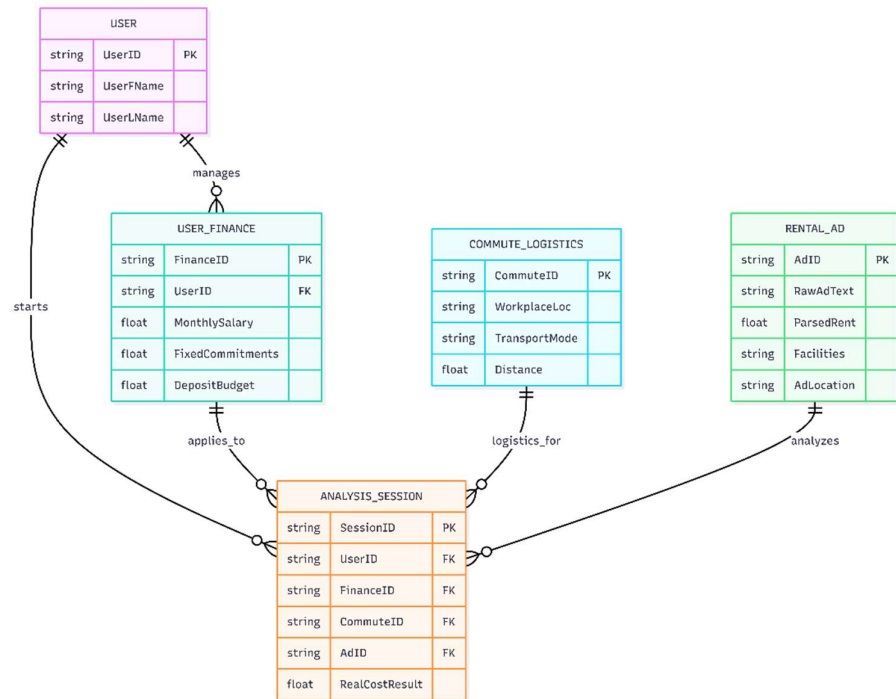


Figure 7: 2 Normal Form

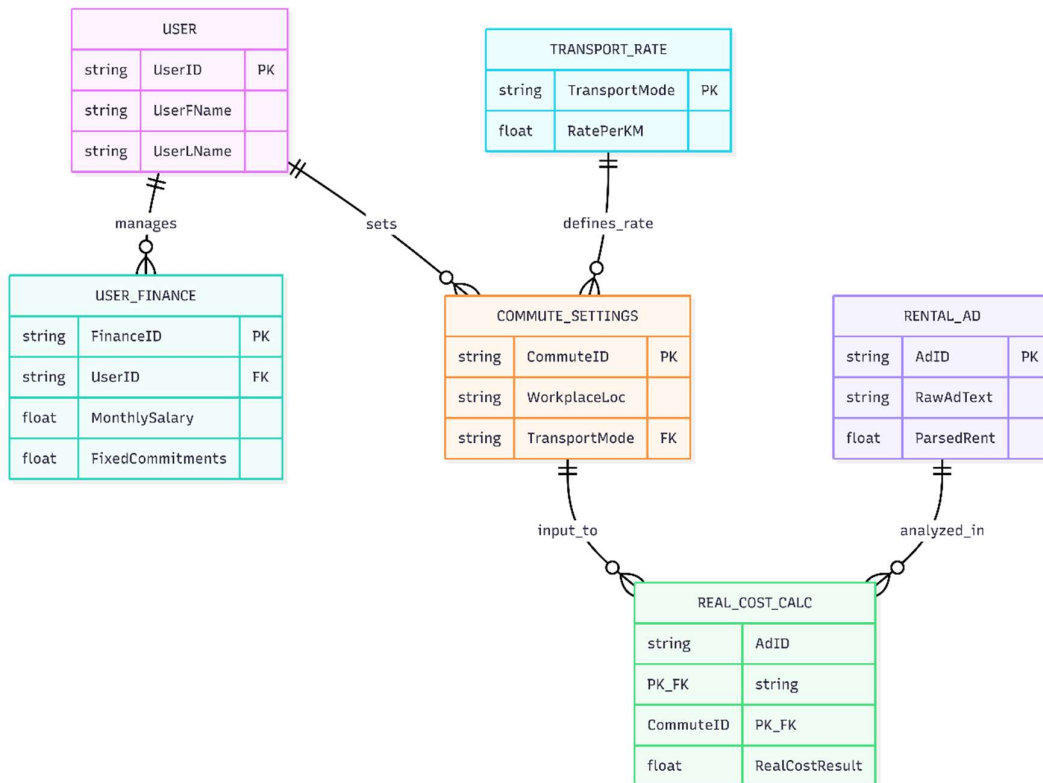


Figure 8: 3 Normal Form

3. Functional Requirements & Scope

This section highlights the core boundaries of SmartRent AI. These boundaries allow the project to focus on a high-impact MVP to showcase technical feasibility during the UMHackathon 2026.

3.1. Minimum Viable Product (MVP)

	Features	Description
1	Financial Profiling	End User inputs monthly salary and fixed commitments to establish a baseline for the Financial Profile.
2	Unstructured Ad Parsing	Z.AI GLM API extracts rent price and facilities from raw Rental Ad Data pasted by the End User.
3	Multi-Option Ranking	SmartRent AI compares up to five rental options and ranks them on the Comparison Canvas based on "Real Cost".
4	Conversational Logic Editing	An AI-driven interface that parses user intent to modify Commute Logistics or financial variables instantly on the 2D canvas

3.2. Non-Functional Requirements (NFRs)

Quality	Requirements	Implementation
Token Latency	The system must return a completed analysis within 45-60 seconds for multi-ad reasoning sessions.	Utilizing a high-fidelity loading overlay ("Z.AI Analysing") to manage user expectations during complex data parsing.
Cost Efficiency	Total tokens used per session must stay within a strict budget to prevent high API costs.	A Token Management Module chunks or truncates the Rental Ad Data to a maximum of 1,500 tokens before sending the prompt.
Data Integrity	The system must handle messy, unstructured text and natural language commands without losing data accuracy.	Using a structured system prompt that forces the Z.AI GLM API to use a standardized mathematical formula for every ad.
Reliability	The system must handle messy or unstructured text without crashing the frontend.	Implementing an "Error Boundary" in React to catch failed API responses and show a graceful fallback message to the End User.

3.3. Out of Scope / Future Enhancements

These features are part of the "Grand Vision" for SmartRent AI but are not included in the UMHackathon 2026 MVP to ensure the team can focus on the core decision intelligence.

- User Authentication & Persistent Database: The current version is stateless. Features like End User login and a persistent SQL database (to save history across devices) are planned for the next version.
- Google Maps API (Real-time GPS): Instead of real-time GPS tracking, the MVP uses manual input for workplace location and distance within the Commute Logistics section.
- Property Owner Dashboard: A portal for landlords to upload ads directly into the system instead of the user pasting ad text manually.
- In-App Messaging: Direct chat functionality between the End User and the property agent/owner.

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation

4.1.1. Grayscale Rollout & A/B Testing

New features or updated AI prompts are released to only 5% of End Users initially. We monitor for errors or "hallucinations" before rolling out to 100%. After a stable release, we conduct A/B testing on different Comparison Canvas layouts to see which visual style helps users make decisions faster.

4.1.2. Emergency Rollback (ER) & Golden Release

If the SmartRent AI crashes or the Z.AI GLM API returns consistent errors, an Emergency Rollback is triggered. The system will automatically redeploy the Golden Version (the last 100% stable version) to ensure the app stays online for the judges.

4.1.3. Priority Matrix

Level	Condition	Action
P1 (Critical)	Z.AI GLM API is down or the app won't load.	Trigger Emergency Rollback and display a "System Maintenance" message.
P2 (High)	AI response time takes more than 745-60 seconds.	Optimize the Token Management Module and reduce the prompt complexity to speed up inference.
P3 (Medium)	Small UI bugs or text overlapping on the Comparison Canvas.	Log the issue and fix it in the next scheduled update.

4.2. Monitoring

We plan to monitor the system by tracking API response codes and token usage. If the Z.AI GLM API returns an error or an empty response, the system will instantly flag it. This allows the developer to fix the prompt logic before more users encounter the same issue.

4.3. Assumptions

- End Users have a stable internet connection to communicate with the AI.
- The Rental Ad Data provided by the user contains at least a price and a general location for the AI to analyze.
- The Z.AI GLM API service is active and has enough credits to process requests during the demo.

4.4. External Dependencies

Tools	Purpose	Risks
Z.AI GLM API	Core reasoning engine	High: If the API is down or rate-limited, the core product will fail.
Vercel	Primary hosting platform for the SmartRent AI.	Vercel provides high availability and global edge distribution. Risk is minimal but relies on their serverless infrastructure uptime.

5. Project Management & Team Contributions

5.1. Project Timeline:

Days	Phase	Activities
Day 1	Kick-off	Registration & Domain Reveal. Initial brainstorming on "Economic Blindness."
Day 2 - 3	Architecture	Finalizing Use Cases. Drafting the SAD and drawing the Diagrams (DFD, ERD, Sequence).
Day 4	UI Coding	Setting up React.js. Building the SmartRent AI and the layout for the Comparison Canvas.
Day 5	API Release	Organizer API Key Release. Configuring the connection between SmartRent AI and the Z.AI GLM API.
Day 6 - 7	AI Logic	Prompt engineering to ensure the AI understands messy ads and calculates the "Real Cost" correctly.
Day 8	Testing	Comprehensive Testing. Squashing bugs, checking AI response speed, and fixing UI alignment.
Day 9	Pitching	Preparing the Pitch Presentation deck and recording the Prototype Demo video.
Day 10	Submission	Final deployment on Vercel and submitting all documentation before the deadline.

5.2 Team Members Role:

Name	Role	Key Responsibilities
Muhammad Aidil	Documentation Lead	Writing the PRD and SAD. Ensuring all technical wording is consistent.
Tania Danisha Puteri	Coding Lead	Developing the SmartRent UI using Vanilla JavaScript and managing Serverless Function logic.
Nur Aqilah	Diagram Specialist	Creating all technical visuals: DFD, ERD, Sequence, and Dependency Diagrams.
Muhammad Nur Eizwan	AI & Prompt Strategist	Designing prompts for the Z.AI GLM and ensuring the Node.js relay accurately parses unstructured rental data.
Nur Farah Hanim	Tester & Presenter	Handling Quality Assurance (QA) testing and preparing the final Pitch Presentation.

5.3. Recommendations

- **Local State Persistence (Browser Auto-Save):** To improve user retention and session continuity, the system should implement `localStorage` to automatically cache the Financial Profile. This ensures that if the page is refreshed or closed accidentally, the user's data remains available, eliminating the need for repetitive manual entry.
- **Advanced PDF Report & Layout Optimization:** While the current MVP supports a basic PDF export, we recommend migrating to a professional library like Puppeteer or React-PDF in the next phase. Implementing CSS Paged Media and print-specific stylesheets will ensure the Comparison Canvas is formatted as a high-fidelity, presentation-ready report with proper headers, branding, and optimized table alignment.
- **Global Session Management (Reset Logic):** A "Global Reset" button is recommended to allow users to clear the current analysis context instantly. This would flush the Z.AI prompt history and reset all financial variables, allowing the user to start a completely fresh property comparison without data remnants from previous sessions.
- **Asynchronous Processing Feedback:** To manage user expectations during the Z.AI reasoning phase (which can exceed 60 seconds), the system should implement a dynamic progress indicator. Instead of a static loading bar, a sequence of live status updates (e.g., "Parsing Rental Features..." → "Calculating Transport Logic...") would significantly improve the perceived performance.